NMIMS Mumbai — Mukesh Patel School of Technology Management & Engineering

B.Tech Computer Engineering — Software Engineering

Software Engineering Mini Project Report

Student Recommendation Platform for New City Students

Project team	Roll number
Krishang Vikram Singh	B106
Samarendra Padaki	B067
Riya Saitwal	B081

Details	
Batch (Krishang Vikram Singh)	C1
Programme	B.Tech Computer Engineering
Subject	Software Engineering
Faculty / guide	[to be filled in]
Date of submission	[to be filled in]

Source code, tests, diagrams and evidence are in the project folder student-recommendation-platform (see README.md).

1. Introduction, problem statement and objectives

Students who move to a new city for college must quickly find a place to live, somewhere to eat, a stationery shop, a place to play sport and things to do. Advice comes from scattered word of mouth, and online reviews are anonymous and hard to trust. The Student Recommendation Platform is a web application where verified students of a college rate and review such places, and an administrator moderates what is published. It was developed across ten Software Engineering experiments (requirements, use cases, activity, class, sequence, collaboration and data-flow design, testing and corrective coding) and this report presents the resulting working system.

Problem statement

New students lack a trustworthy, location-aware source of information about accommodation, food, stationery, sport and events near their campus, and existing review sites do not confirm that a reviewer is a student of the college or check what is published.

Objectives

- Let students register with a college email and log in securely, with lockout after repeated failures.
- Let anyone search nearby places by category, filter by cost, rating and distance, and browse upcoming events.
- Let only verified students submit 1–5 star reviews, and publish them only after administrator approval, so that average ratings reflect approved reviews only.
- Give administrators tools to moderate reviews and reports and to manage students, colleges, cities and listings, with an audit trail.
- Verify the system with automated tests, a live smoke test, and measurements against the quality targets of Experiment 2.

Scope

In scope: the five categories (PGs/hostels, restaurants, stationery shops, sports facilities, events), college-domain verification, reviews, reports, notifications, personalised recommendations and the administrator functions above. Out of scope (not built): HTTPS hosting, email delivery, maps or third-party business data, college-ID verification, slot booking and other Experiment 5 operations that have no user story. All listings in the demonstration are sample data.

2. Requirements summary

Requirements come from Experiment 1 (eleven Agile user stories, US1–US11) and Experiment 2 (quality targets). Experiment 10 added four business rules found through testing. The full inventory, traceability matrix and a list of 17 contradictions between the experiments are in the docs folder.

Table 1. Functional requirements and their status

ID	Functional requirement (Experiment 1)	Status
US1–US2	Register with college email; duplicate and non-college emails rejected; account marked verified	Done; domain check only
US3	Login, error message, lock for 15 minutes after 5 consecutive failures	Done
US4, US9	Search within 5 km sorted by distance; filters for cost, rating, distance; empty-result guidance	Done
US5	Events in date order with type and date filters	Done
US6–US7	Verified students review and report; reports make the review “Under Review”; no duplicate reports	Done; publication after approval

US8	Personalised recommendations from saved preferences and location	Done
US10	Notifications with read state and per-category opt-out	Done (in-app)
US11	Admin moderation, suspension and reinstatement, logged	Done; no edit-flag action

Table 2. Non-functional targets

Quality attribute	Target (Experiment 2)
Performance	Non-search ≤ 1 s; search ≤ 3 s
Availability	99.5 % per month
Security	Authenticated, encrypted sessions; failed attempts limited and logged
Retention / reliability	Reviews kept ≥ 2 years; 99.9 % of submissions recorded correctly
Usability	Register and search in ≤ 5 clicks
Recoverability	Service restored within 8 h
Scalability	Search ≤ 3 s with 100 000 users and 500 concurrent sessions
Confidentiality, integrity	No private details exposed; invalid ratings, empty fields and spam rejected
Portability, accessibility	Chrome/Edge/Safari and phone sizes; WCAG 2.1 AA

3. System architecture and design

The system is a small layered monolith (Figure 1). A browser single-page interface calls a JSON API implemented with the Python standard library HTTP server. All business rules live in one service class that is independent of HTTP. Working data is held in memory and every change is written through to a SQLite database whose schema enforces keys, unique emails and the 1–5 rating range. Python was chosen because the earlier experiments already described Python logic and pytest tests; it keeps the project dependency-free and easy to demonstrate.

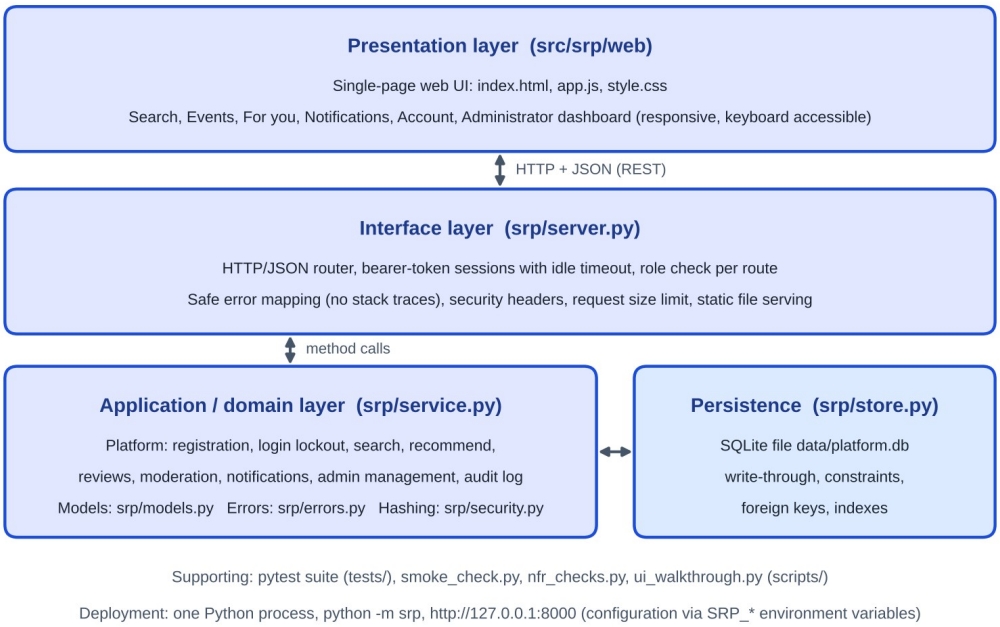


Figure 1. Architecture of the implemented system

Table 3. Modules

Module	Responsibility
web/ (HTML, CSS, JavaScript)	Search, events, recommendations, notifications, account and administrator screens
server.py	Routing, bearer-token sessions with 30-minute idle timeout, role checks, safe error messages, security headers

service.py	Registration, login lockout, search, recommendations, reviews, moderation, notifications, administration, audit log
models.py, errors.py, security.py	Domain objects, error types, scrypt password hashing
store.py	SQLite schema (users, colleges, cities, recommendations, reviews, reports, notifications, audit log) and write-through saves

3.1 Use cases and class design

The use-case diagram (Figure 2) shows the Student and Administrator actors. The original Experiment 3 image was not available for this build, so the diagram was redrawn from Experiments 1, 4 and 5 and is labelled as reconstructed. The class diagram (Figure 3) is the one drawn in Experiment 5: an abstract User with Student and Administrator, an abstract Recommendation with five subclasses, College, City and Review.

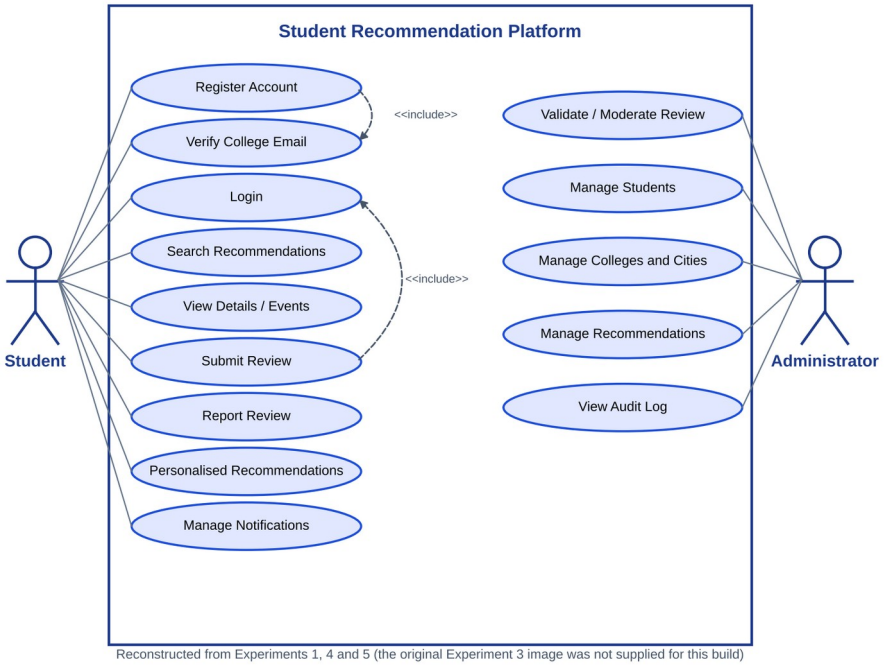


Figure 2. Use-case diagram (reconstructed from Experiments 1, 4 and 5)

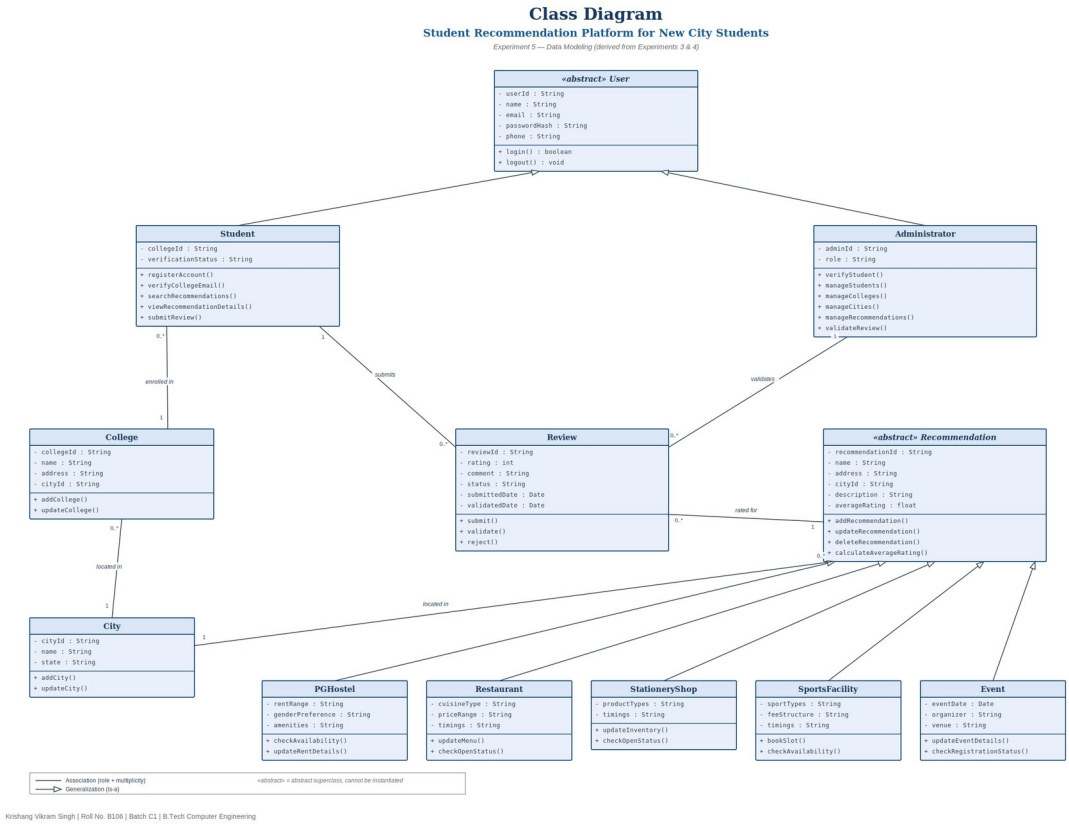


Figure 3. Class diagram from Experiment 5

Differences between the class diagram and the code are listed in the consistency analysis. In short: one users table with a role column replaces the two user subclasses; the category-specific attributes (rent range, cuisine, timings, sport types, organizer, venue) are stored in a JSON details column; Notification, Report and Audit entries were added because US7, US10 and Experiment 2 need them; operations such as bookSlot and updateMenu have no user story and were not built.

3.2 Workflow and interaction design

Figure 4 (activity diagram, Experiment 4) and Figure 5 (sequence diagram, Experiment 6) show the main path: register, verify, log in, search, view details, submit a review, administrator validation and notification.

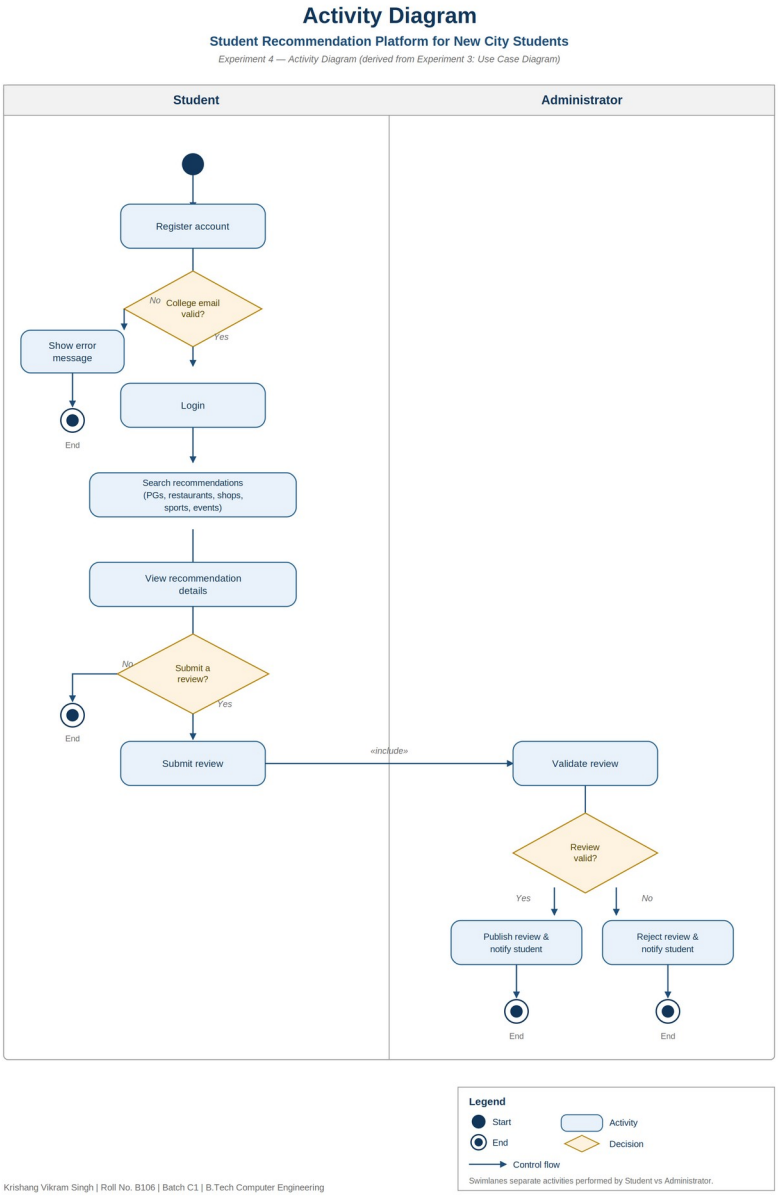


Figure 4. Activity diagram with Student and Administrator swimlanes (Experiment 4)

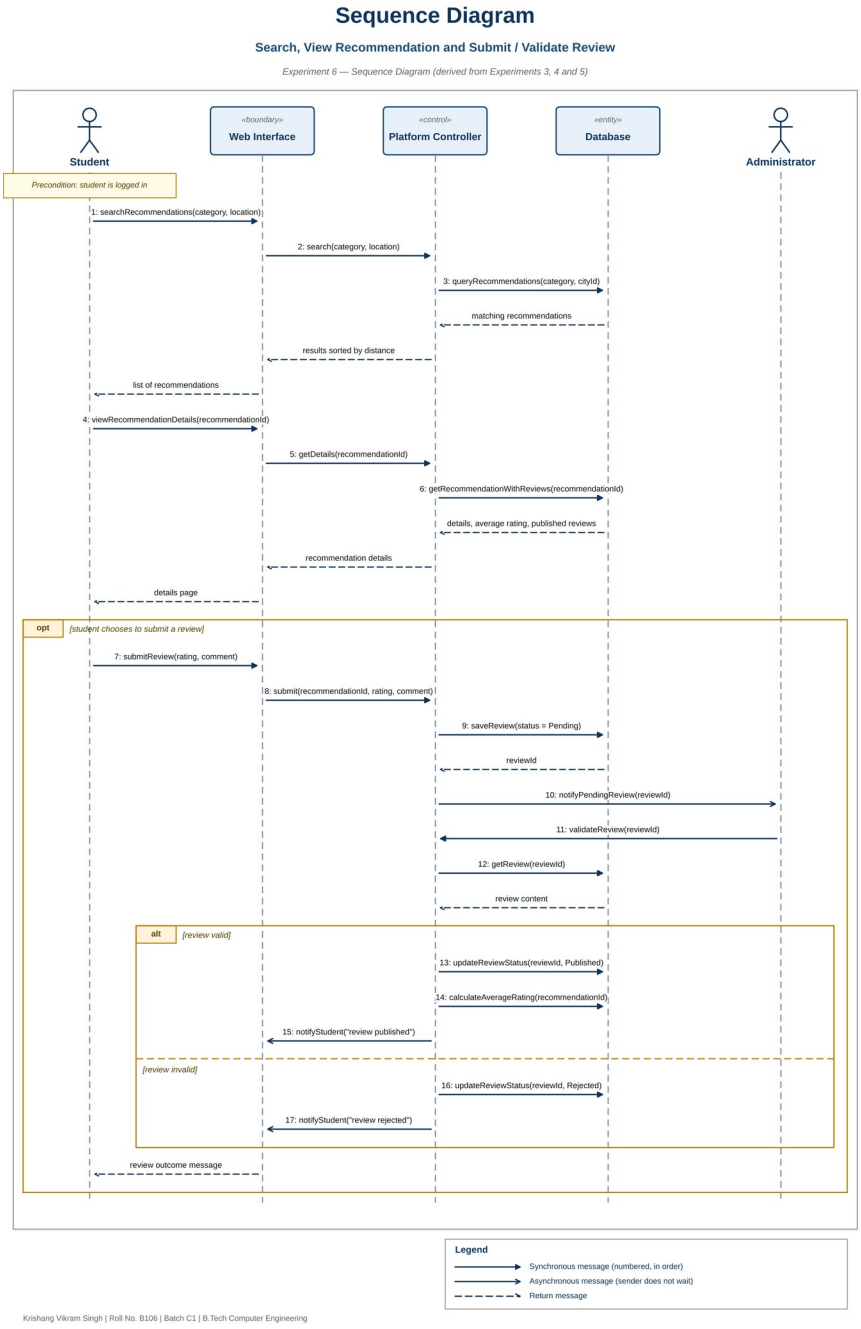


Figure 5. Sequence diagram: search, view, submit and validate a review (Experiment 6)

3.3 Data flow

The Level 0 and Level 1 data-flow diagrams (Experiment 8) show four processes — manage accounts, search and recommend, manage reviews, administer platform data — and four data stores (users, colleges and cities, recommendations, reviews). These correspond to the registration/login, search, review and administration methods of the service class and the matching tables in the database.

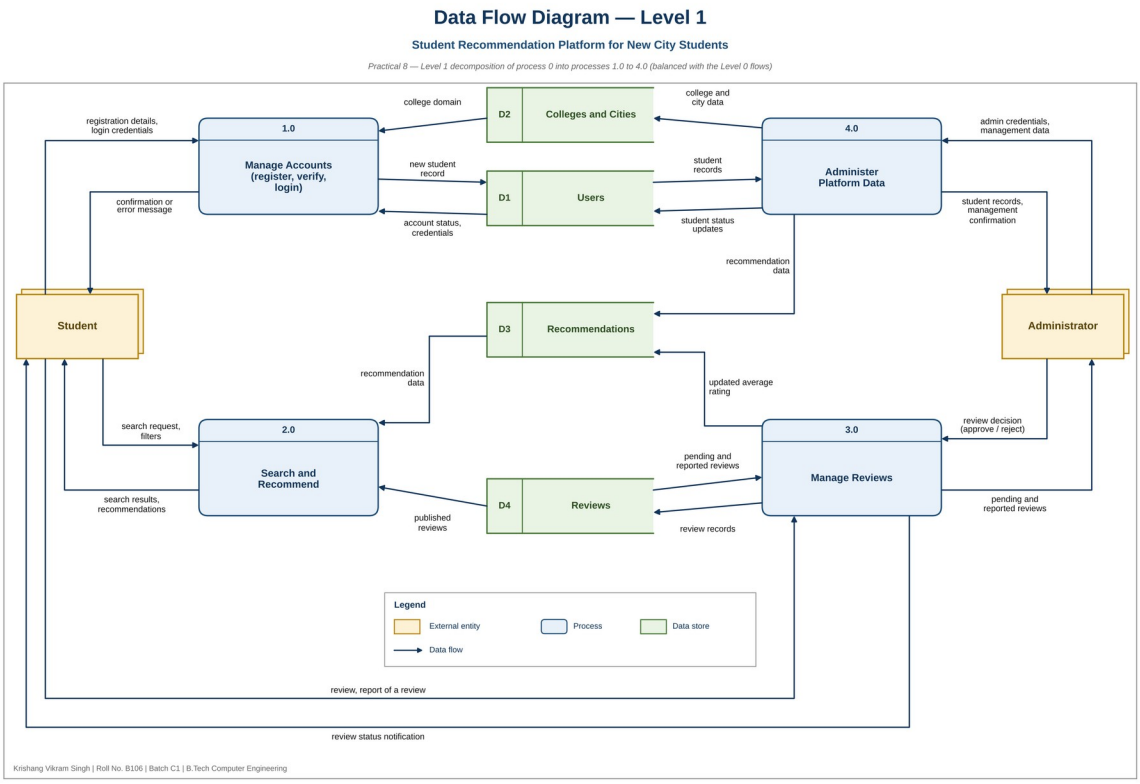


Figure 6. Data flow diagram, Level 1 (Experiment 8)

4. Implementation

The code is about 1 400 lines of Python and a 260-line JavaScript interface, with no third-party runtime packages. Key rules:

- Registration: the email domain must equal the college domain or be a subdomain of it, so “notnmims.edu.in” is rejected; emails are lower-cased before comparison; passwords need 8 or more characters and are stored as salted scrypt hashes.
- Login: five consecutive failures lock the account for 15 minutes; the lock expires by itself; suspended accounts are refused and their open sessions are ended; every failure is written to the audit log.
- Search: haversine distance, default radius 5 km, filters for cost and rating, results sorted by distance and limited to 50; events are hidden once their date has passed.
- Reviews: integer rating 1–5 (booleans and fractions rejected), mandatory comment up to 1 000 characters, URL/repeat/profanity screening, one review per student per listing; reviews start Pending and count towards the average only when Published; a reported review leaves the average until the administrator resolves it.
- Administration: every administrator method checks the role on the server; catalogue edits, verification, suspension and moderation are logged with actor, time and target; listings are archived rather than deleted so reviews are retained.
- Confidentiality: public responses contain display names and review text only; the administrator list shows masked emails.

A limitation worth stating plainly: verification is by email domain. The platform sends no confirmation email, so it cannot prove that the student owns the mailbox; an administrator can verify students manually.

5. Testing methodology and results

Testing combined pytest unit and API tests, a live smoke test, a browser walkthrough and load measurements. The first prototype (Experiment 9) failed 4 of 26 tests; the fixes of Experiment 10 made all 26 pass, and no test was weakened afterwards. The suite was then extended to 96 tests.

Table 4. Test summary

Check	Command / tool	Result
Automated tests	python -m pytest -v	96 passed, 0 failed; 92 % line coverage of src/srp
Core cases TC-01 to TC-04	tests/test_case_study.py	All pass: non-college email rejected; lock after 5 failures; unverified student blocked; approved reviews give 4.5
Historical defects D-1 to D-4	tests/test_extended.py, test_features.py	All pass: look-alike domain, case-different duplicate, suspended login, 4.5 and True ratings rejected
Live smoke test	scripts/smoke_check.py	18 checks passed against the running server (201/422/409/403/423/401 statuses as expected)
Restart	stop and start on the same database	Accounts, lock state, review and rating preserved
Browser walkthrough	Playwright, Chromium	Register, search, review, approve and recommend flow completed; 0 axe-core violations on 7 screens

Quality targets: measured results

Table 5. Experiment 2 targets

Attribute	Measured (one 4-core machine, loopback)	Verdict
Performance	Search median 6 ms, login median 51 ms (100 001 users, 20 000 reviews)	Met for single requests
Burst of logins	20 parallel logins: median 1.09 s, max 1.90 s (hashing runs one at a time)	1 s target missed in this burst
Scalability	500 sessions, 500 simultaneous searches: all succeeded, max 1.94 s	Met for search
Reliability	500/500 simultaneous reviews stored, 500 database rows	Met in this test
Recoverability	Restart healthy in 2.06 s with all data	Process restart only
Usability	3 clicks to register and search	Met
Accessibility / portability	axe-core: 0 violations; Chromium desktop and 390 px phone only	Partly met
Availability, encrypted transport	Not measured; plain HTTP only	Not verified / not met

These figures describe this environment only. Availability and long-term retention cannot be established by tests run in a single session; HTTPS was not implemented; portability and accessibility were checked only with Chromium and an automated scanner.

6. Screenshots and demonstration results

The screens below were captured from the running application by the walkthrough script (sample data).

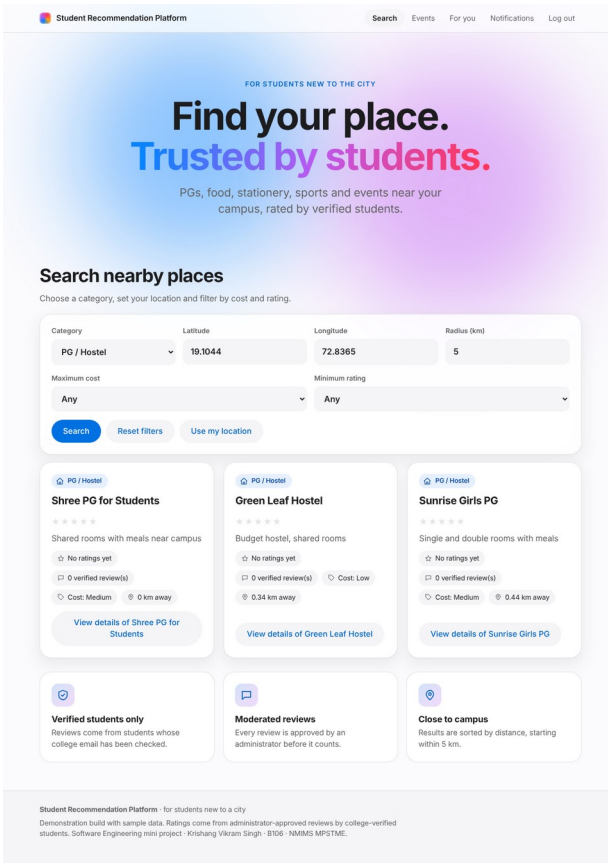


Figure 7. Search results with rating, cost, distance and review count

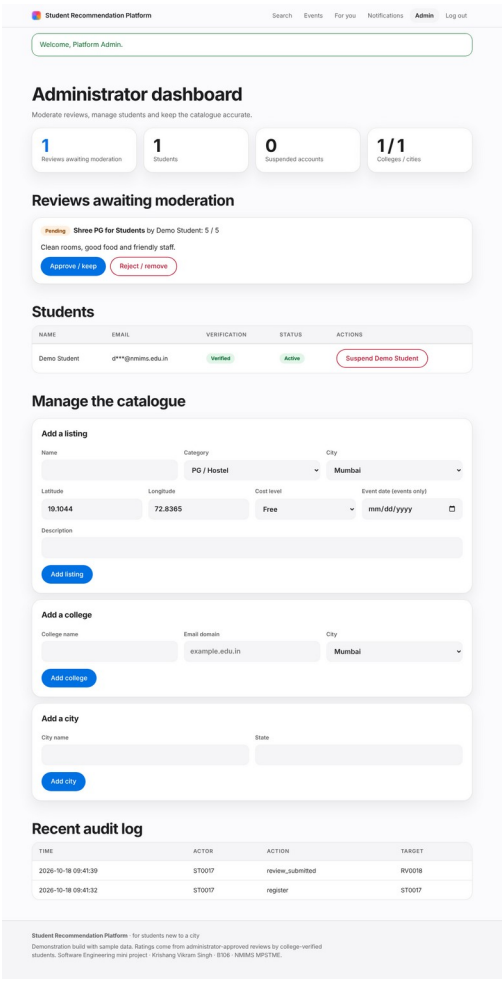


Figure 8. Administrator dashboard with the pending review

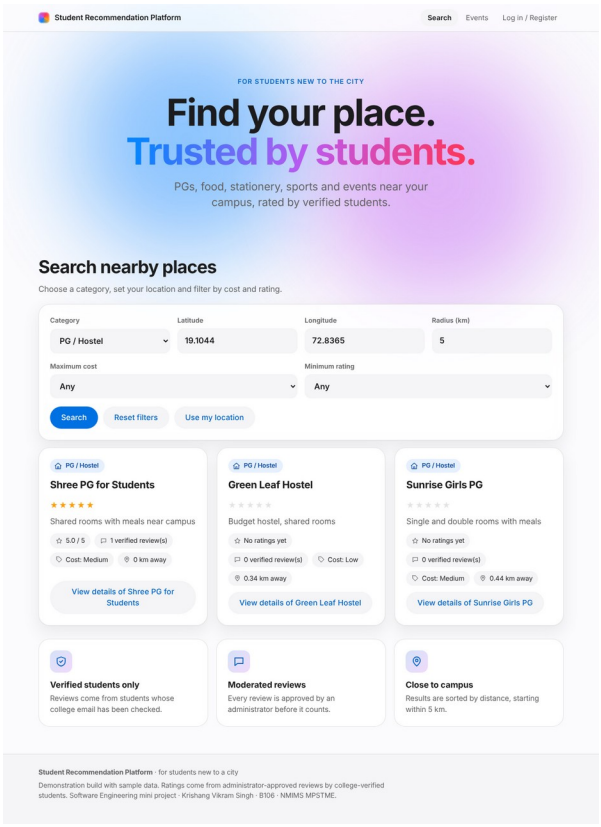


Figure 9. After approval the rating and review count update

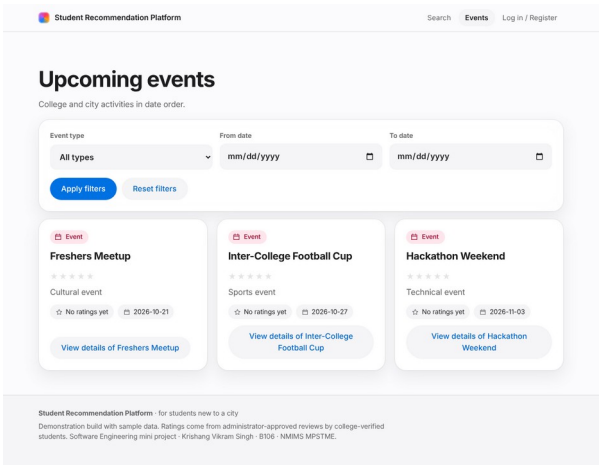


Figure 10. Events in chronological order

7. Deployment

The application runs as one process: set PYTHONPATH=src and SRP_ADMIN_PASSWORD, then run python -m srp. It listens on http://127.0.0.1:8000/, creates data/platform.db and loads sample data on first run. This local deployment was started, exercised by the smoke test and restarted to confirm that data persists. No public hosting exists; the README explains how to add a TLS proxy before any public use.

8. Limitations and future enhancements

- No HTTPS; email ownership and college ID are not verified (confirmation links would fix the first).
- Publication needs administrator approval, so a review does not appear within one minute as US6 words it.
- “Edit-flag” moderation, slot booking, menu and inventory updates, push or email notifications and real maps are not built.
- Single process, in-memory sessions, no backup or archival job; login bursts exceeded 1 s because password hashing is serialised.
- Availability, long-term retention, Safari/Edge/Firefox and manual screen-reader testing remain unverified.
- Future work: confirmation emails, TLS deployment, a hosted database, per-IP rate limiting, cached search index, and a user study.

9. Conclusion

The project turned the requirements and designs of Experiments 1–8 into a working, persistent and tested web application. All eleven user stories are implemented except where noted, the four defects found in testing are fixed and guarded by regression tests, and 96 automated tests pass. Measured performance meets the search

and scalability targets on one machine, while encrypted transport, availability, broader browser coverage and manual accessibility checks are still open. The traceability matrix and consistency analysis make clear which claims are backed by evidence and which are not.

References

1. K. V. Singh, Software Engineering Lab Record, Experiments 1–10 (B106, Batch C1), NMIMS MPSTME, 2026 (requirements, use case, activity, class, sequence, collaboration, data-flow, testing and coding experiments).
2. M. Cohn, User Stories Applied: For Agile Software Development, Addison-Wesley, 2004.
3. M. Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language, 3rd ed., Addison-Wesley, 2003.
4. C. Gane and T. Sarson, Structured Systems Analysis: Tools and Techniques, Prentice-Hall, 1979.
5. I. Sommerville, Software Engineering, 10th ed., Pearson, 2016.
6. R. W. Sinnott, “Virtues of the Haversine,” Sky and Telescope, vol. 68, no. 2, p. 159, 1984.
7. W3C, Web Content Accessibility Guidelines (WCAG) 2.1, https://www.w3.org/TR/WCAG21/
8. Python Software Foundation, Python 3 documentation: sqlite3, http.server, hashlib.scrypt, https://docs.python.org/3/
9. pytest documentation, https://docs.pytest.org/ ; Deque Systems, axe-core, https://github.com/dequelabs/axe-core ; Playwright, https://playwright.dev/